

```

# -*- coding: utf-8 -*-
"""بدون خلفيه.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1S-iXi8Bie0JeUE3UyIU
"""

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from collections import defaultdict

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (accuracy_score, precision_score, re
                             f1_score, confusion_matrix, classif
                             roc_auc_score, roc_curve)

import warnings
warnings.filterwarnings('ignore')

from google.colab import drive
drive.mount('/content/drive')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"✅ Libraries imported")
print(f"    Device: {device}")
if device.type == 'cuda':
    print(f"    GPU: {torch.cuda.get_device_name(0)}")

# =====
# الخلية 2: تحميل الصور (بدون خلفية) - كل البراندات
# =====

BASE = '/content/drive/MyDrive/GP2 (أصل)/Data/product_images/'

```

```

# — المجلدات لكل البراندات —
folders = {
    'chanel_fake': os.path.join(BASE, 'Chanel/replica_nobg_chanel_fake.jpg'),
    'chanel_real': os.path.join(BASE, 'Chanel/original_nobg_chanel_real.jpg'),
    'coach_fake': os.path.join(BASE, 'صور كوتش/replica_nobg_coach_fake.jpg'),
    'coach_real': os.path.join(BASE, 'صور كوتش/original_nobg_coach_real.jpg'),
    'louisvuitton_fake': os.path.join(BASE, 'Louis Vuitton/replica_nobg_louisvuitton_fake.jpg'),
    'louisvuitton_real': os.path.join(BASE, 'Louis Vuitton/original_nobg_louisvuitton_real.jpg'),
    'ysl_fake': os.path.join(BASE, 'YSL/replica_nobg_ysl_fake.jpg'),
    'ysl_real': os.path.join(BASE, 'YSL/original_nobg_ysl_real.jpg'),
    'prada_fake': os.path.join(BASE, 'prada/replica_nobg_prada_fake.jpg'),
    'prada_real': os.path.join(BASE, 'prada/real_nobg_prada_real.jpg'),
    'fendi_fake': os.path.join(BASE, 'fendi/replica_nobg_fendi_fake.jpg'),
    'fendi_real': os.path.join(BASE, 'fendi/real_nobg_fendi_real.jpg')
}

def get_brand(key):
    if 'chanel' in key: return 'Chanel'
    if 'coach' in key: return 'Coach'
    if 'louisvuitton' in key: return 'Louis Vuitton'
    if 'ysl' in key: return 'YSL'
    if 'prada' in key: return 'Prada'
    if 'fendi' in key: return 'Fendi'

image_data = []

for key, folder_path in folders.items():
    brand = get_brand(key)
    is_authentic = 1 if 'real' in key else 0

    if not os.path.exists(folder_path):
        print(f"⚠ Folder not found: {folder_path}")
        continue

    files = sorted([f for f in os.listdir(folder_path)
                    if f.lower().endswith(('.jpg', '.jpeg', '.png'))])

    for f in files:
        filepath = os.path.join(folder_path, f)
        parts = f.replace('.jpg', '').replace('.jpeg', '').replace('.png', '')

        product_id = None
        for p in parts:
            try:
                num = int(p)
            except:
                continue

```

```

        if product_id is None:
            product_id = num
    except:
        continue
    if product_id is None:
        product_id = f

    image_data.append({
        'filepath':    filepath,
        'filename':    f,
        'product_id':  f"{brand}_{is_authentic}_{product_id}",
        'brand':       brand,
        'is_authentic': is_authentic,
    })

    print(f"  ✅ {key}: {len(files)} images")

df_images = pd.DataFrame(image_data)

product_groups = df_images.groupby('product_id').agg({
    'filepath':    list,
    'is_authentic': 'first',
    'brand':       'first',
}).reset_index()

print(f"\n🇮🇹 Summary (No Background Images – All Brands):")
print(f"  Total images   : {len(df_images)}")
print(f"  Total products: {len(product_groups)}")
for brand in ['Chanel', 'Coach', 'Louis Vuitton', 'YSL', 'Prada']:
    b = product_groups[product_groups['brand'] == brand]
    print(f"    {brand:<15}: {len(b)} products "
          f"(Fake={len(b[b['is_authentic']==0])}   Real={len(b[b['is_authentic']==1])}")

# =====
# 3 الخلية: Dataset Class
# =====

class BagImageDataset(Dataset):
    def __init__(self, product_ids, labels, image_paths_dict, transform):
        self.product_ids = product_ids
        self.labels       = labels
        self.image_paths_dict = image_paths_dict
        self.transform     = transform

    def __len__(self):

```

```

    def __len__(self):
        return len(self.product_ids)

    def __getitem__(self, idx):
        pid = self.product_ids[idx]
        label = self.labels[idx]
        paths = self.image_paths_dict[pid]
        img_path = paths[np.random.randint(len(paths))]

        try:
            image = Image.open(img_path).convert('RGB')
        except:
            image = Image.new('RGB', (224, 224), (0, 0, 0))

        if self.transform:
            image = self.transform(image)

        return image, torch.tensor(label, dtype=torch.float32)

train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.229, 0.229]),
])

test_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.229, 0.229]),
])

print("✅ Dataset class & transforms defined")

# =====
# الخلية 4: بناء EfficientNet-B0
# =====

def create_efficientnet():
    model = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
    for param in model.parameters():
        param.requires_grad = False

```

```

        param.requires_grad = False
    for param in model.features[-3:].parameters():
        param.requires_grad = True
    model.classifier = nn.Sequential(
        nn.Dropout(0.4),
        nn.Linear(1280, 256),
        nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(256, 1),
        nn.Sigmoid()
    )
    return model

model_test = create_efficientnet()
trainable = sum(p.numel() for p in model_test.parameters() if p.requires_grad)
total = sum(p.numel() for p in model_test.parameters())
print(f"✅ EfficientNet-B0: {trainable:,} trainable / {total:,} total")
del model_test

# =====
# الخلية 5: دوال التدريب والتقييم
# =====

def train_image_model(model, train_loader, val_loader, epochs=30):
    criterion = nn.BCELoss()
    optimizer = torch.optim.AdamW(filter(lambda p: p.requires_grad, model.parameters()), lr=lr, weight_decay=1e-4)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, 'min')

    history = {'train_loss': [], 'val_loss': [], 'train_acc': []}
    best_val_loss = float('inf')
    best_state = None
    patience_counter = 0

    for epoch in range(epochs):
        model.train()
        t_loss, t_correct, t_total = 0, 0, 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images).squeeze(-1)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            t_loss += loss.item()

```

```

        t_correct += ((outputs >= 0.5).float() == labels).sum()
        t_total   += len(labels)

model.eval()
v_loss, v_correct, v_total = 0, 0, 0
with torch.no_grad():
    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images).squeeze(-1)
        v_loss += criterion(outputs, labels).item()
        v_correct += ((outputs >= 0.5).float() == labels).sum()
        v_total   += len(labels)

avg_tl = t_loss / len(train_loader)
avg_vl = v_loss / len(val_loader)
t_acc  = t_correct / t_total
v_acc  = v_correct / v_total

history['train_loss'].append(avg_tl)
history['val_loss'].append(avg_vl)
history['train_acc'].append(t_acc)
history['val_acc'].append(v_acc)
scheduler.step(avg_vl)

marker = ""
if avg_vl < best_val_loss:
    best_val_loss = avg_vl
    best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
    patience_counter = 0
    marker = "✅"
else:
    patience_counter += 1

print(f"    Epoch {epoch+1:>2}/{epochs} | TrLoss:{avg_tl:.4f} | ValLoss:{avg_vl:.4f} | Patience:{patience_counter}")

if patience_counter >= patience:
    print(f"    🛑 Early stopping at epoch {epoch+1}")
    break

model.load_state_dict(best_state)
model.to(device)
return model, history

def evaluate_image_model(model, test_loader):
    model.eval()

```

```

    preds, probs, labels = [], [], []
    with torch.no_grad():
        for images, lbls in test_loader:
            images = images.to(device)
            outputs = model(images).squeeze()
            preds.extend((outputs >= 0.5).float().cpu().numpy())
            probs.extend(outputs.cpu().numpy())
            labels.extend(lbls.numpy())
    return np.array(preds), np.array(probs), np.array(labels)

print("✅ Training functions defined")

# =====
# 5 :6 -Fold CV - البراندات مجتمعين - الخليه
# =====

N_FOLDS = 5
EPOCHS = 30
BATCH_SIZE = 16

product_ids = product_groups['product_id'].values
product_labels = product_groups['is_authentic'].values
product_brands = product_groups['brand'].values
stratify_col = [f"{b}_{l}" for b, l in zip(product_brands, product_labels)]

image_paths_dict = dict(zip(product_groups['product_id'], product_groups['image_path']))

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=42)

all_fold_results = []
all_true, all_pred, all_prob = [], [], []
fold_histories = []
best_states = []

print(f"🚀 Starting {N_FOLDS}-Fold CV (All Brands - No Background Noise)")
print(f"Products: {len(product_ids)} | Images: {len(df_images)}")
print("="*70)

for fold, (train_idx, test_idx) in enumerate(skf.split(product_ids, stratify_col)):
    print(f"\n{'='*60}")
    print(f"📌 FOLD {fold+1}/{N_FOLDS}")
    print(f"{'='*60}")

    train_pids = product_ids[train_idx]
    test_pids = product_ids[test_idx]

```

```

train_labels = product_labels[train_idx]
test_labels  = product_labels[test_idx]

val_size      = int(0.15 * len(train_pids))
val_pids      = train_pids[-val_size:]
val_labels_split = train_labels[-val_size:]
train_pids_split = train_pids[:-val_size]
train_labels_split = train_labels[:-val_size]

print(f"    Train: {len(train_pids_split)} | Val: {len(val_pids_split)}")

train_ds = BagImageDataset(train_pids_split, train_labels_split)
val_ds   = BagImageDataset(val_pids,        val_labels_split)
test_ds  = BagImageDataset(test_pids,       test_labels,

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
val_loader   = DataLoader(val_ds,   batch_size=BATCH_SIZE, shuffle=False)
test_loader  = DataLoader(test_ds,  batch_size=BATCH_SIZE, shuffle=False)

model, history = train_image_model(create_efficientnet().to(device), train_loader, val_loader, test_loader)
fold_histories.append(history)

fold_probs_list = []
for _ in range(3):
    _, probs, _ = evaluate_image_model(model, test_loader)
    fold_probs_list.append(probs)

avg_probs = np.mean(fold_probs_list, axis=0)
final_preds = (avg_probs >= 0.5).astype(float)

acc = accuracy_score(test_labels, final_preds)
prec = precision_score(test_labels, final_preds, zero_division=0)
rec = recall_score(test_labels, final_preds, zero_division=0)
f1 = f1_score(test_labels, final_preds, zero_division=0)
auc = roc_auc_score(test_labels, avg_probs)

all_fold_results.append({'Fold': fold+1, 'Accuracy': acc, 'Precision': prec, 'Recall': rec, 'F1': f1, 'AUC': auc})
all_true.extend(test_labels)
all_pred.extend(final_preds)
all_prob.extend(avg_probs)

print(f"\n    📊 Fold {fold+1}: Acc={acc:.3f} Prec={prec:.3f} Rec={rec:.3f} F1={f1:.3f} AUC={auc:.3f}")

best_states.append({k: v.cpu().clone() for k, v in model.state_dict().items()})

```



```
# 1 <= i <= n, j <= i ~ 1 ~ i product      & test = 0, n, d, i, j, skf
```

```

# =====
brand_per_test = []
for fold, (train_idx, test_idx) in enumerate(skf.split(product_ids)):
    test_pids = product_ids[test_idx]
    for pid in test_pids:
        b = product_groups[product_groups['product_id'] == pid]
        brand_per_test.append(b)

brand_per_test = np.array(brand_per_test)

print(f"\n{'Brand':<18} {'Acc':>8} {'Prec':>8} {'Rec':>8} {'F1':>8}")
print("-" * 68)

for brand in ['Chanel', 'Coach', 'Louis Vuitton', 'YSL', 'Prada']:
    mask = brand_per_test == brand
    if mask.sum() == 0:
        continue
    bt = all_true_arr[mask]
    bp = all_pred_arr[mask]
    bprob = all_prob_arr[mask]
    try:
        auc_b = roc_auc_score(bt, bprob)
    except:
        auc_b = float('nan')
    print(f" {brand:<16} {accuracy_score(bt, bp):>8.4f} "
          f"{precision_score(bt, bp, zero_division=0):>8.4f} "
          f"{recall_score(bt, bp, zero_division=0):>8.4f} "
          f"{f1_score(bt, bp, zero_division=0):>8.4f} "
          f"{auc_b:>8.4f} {mask.sum():>6}")

# =====
# 9 الخلية: Confusion Matrix
# =====

cm = confusion_matrix(all_true, all_pred)
acc_all = accuracy_score(all_true, all_pred)
f1_all = f1_score(all_true, all_pred)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, annot_kws={
    'xticklabels=['Fake', 'Real'], 'yticklabels=['Fake', 'Real']})
ax.set_ylabel('Actual', fontsize=13)
ax.set_xlabel('Predicted', fontsize=13)
ax.set_title(f'Exp 2b (All Brands): Images (No Background) – Efficiency of 5-Fold CV | Acc={acc_all:.3f} | F1={f1_all:.3f}',
             fontsize=14, fontweight='bold')

```

```

        fontsize=14, fontweight='bold',
plt.tight_layout()
plt.savefig('exp2b_allbrands_confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()

# =====
# 10 الخلية: ROC Curve - لكل براند
# =====

colors = {'Chanel': '#e74c3c', 'Coach': '#3498db', 'Louis Vuitton': '#2ecc71',
          'Prada': '#f39c12', 'Fendi': '#1abc9c'}

plt.figure(figsize=(9, 7))

# Overall
fpr_all, tpr_all, _ = roc_curve(all_true, all_prob)
auc_all = roc_auc_score(all_true, all_prob)
plt.plot(fpr_all, tpr_all, color='black', linewidth=3,
         label=f'Overall (AUC={auc_all:.3f})', linestyle='--')

# Per brand
for brand, color in colors.items():
    mask = brand_per_test == brand
    if mask.sum() == 0 or len(np.unique(all_true_arr[mask])) < 2:
        continue
    fpr_b, tpr_b, _ = roc_curve(all_true_arr[mask], all_prob_arr[mask])
    auc_b = roc_auc_score(all_true_arr[mask], all_prob_arr[mask])
    plt.plot(fpr_b, tpr_b, color=color, linewidth=2, label=f'{brand} (AUC={auc_b:.3f})')

plt.plot([0, 1], [0, 1], 'k:', linewidth=1, label='Random (0.500)')
plt.fill_between(fpr_all, tpr_all, alpha=0.05, color='black')
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('Exp 2b: ROC Curves - All Brands (No Background)', fontweight='bold',
         fontstyle='italic', fontfamily='serif', fontsize=12)
plt.legend(loc='lower right', fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('exp2b_allbrands_roc_curve.png', dpi=150, bbox_inches='tight')
plt.show()

# =====
# 11 الخلية: Training History
# =====

```

```

fig, axes = plt.subplots(2, N_FOLDS, figsize=(5*N_FOLDS, 8))
fig.suptitle('Exp2b (All Brands): EfficientNet-B0 Training History',
             fontsize=16, fontweight='bold')

for fold_i, hist in enumerate(fold_histories):
    axes[0][fold_i].plot(hist['train_loss'], label='Train', color='red')
    axes[0][fold_i].plot(hist['val_loss'], label='Val', color='blue')
    axes[0][fold_i].set_title(f'Fold {fold_i+1} - Loss', fontweight='bold')
    axes[0][fold_i].set_xlabel('Epoch')
    axes[0][fold_i].legend()
    axes[0][fold_i].grid(True, alpha=0.3)

    axes[1][fold_i].plot(hist['train_acc'], label='Train', color='red')
    axes[1][fold_i].plot(hist['val_acc'], label='Val', color='blue')
    axes[1][fold_i].set_title(f'Fold {fold_i+1} - Accuracy', fontweight='bold')
    axes[1][fold_i].set_xlabel('Epoch')
    axes[1][fold_i].legend()
    axes[1][fold_i].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('exp2b_allbrands_training_history.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Saved: exp2b_allbrands_training_history.png")

# =====
# الخلية 12: حفظ الملفات
# =====

fold_df.to_csv('exp2b_allbrands_results.csv', index=False)
fold_df.to_excel('exp2b_allbrands_results.xlsx', index=False)

print("\n✅ FILES SAVED:")
print("📄 exp2b_allbrands_results.csv / .xlsx")
print("📊 exp2b_allbrands_confusion_matrix.png")
print("📈 exp2b_allbrands_roc_curve.png")
print("📈 exp2b_allbrands_training_history.png")

print("\n" + "="*60)
print("🎉 EXPERIMENT 2b (ALL BRANDS) COMPLETE!")
print("="*60)
print(f"\n🇮🇹 Overall: Acc={fold_df['Accuracy'].mean():.4f} "
      f"F1={fold_df['F1'].mean():.4f} AUC={fold_df['AUC'].mean():.4f}")

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount('/content/drive')

✅ Libraries imported

Device: cuda

```
#####
```

```
GPU: Tesla T4
```

```

✓ chanel_fake: 363 images
✓ chanel_real: 517 images
✓ coach_fake: 388 images
✓ coach_real: 560 images
✓ louisvuitton_fake: 54 images
✓ louisvuitton_real: 396 images
✓ ysl_fake: 592 images
✓ ysl_real: 410 images
✓ prada_fake: 527 images
✓ prada_real: 509 images
✓ fendi_fake: 305 images
✓ fendi_real: 417 images

```



Summary (No Background Images – All Brands):

Total images : 5038

Total products: 1075

Chanel : 188 products (Fake=94 Real=94)

Coach : 188 products (Fake=94 Real=94)

Louis Vuitton : 141 products (Fake=47 Real=94)

YSL : 191 products (Fake=96 Real=95)

Prada : 188 products (Fake=94 Real=94)

Fendi : 179 products (Fake=86 Real=93)



Dataset class & transforms defined



EfficientNet-B0: 3,483,933 trainable / 4,335,741 total (80.4%)



Training functions defined



Starting 5-Fold CV (All Brands – No Background)

Products: 1075 | Images: 5038



FOLD 1/5

```
Train: 731 | Val: 129 | Test: 215
```

```
Epoch 1/30 | TrLoss:0.6863 VLoss:0.6740 | TrAcc:0.522 VAcc:0.5
```

```
Epoch 2/30 | TrLoss:0.6187 VLoss:0.5797 | TrAcc:0.714 VAcc:0.7
```

```
Epoch 3/30 | TrLoss:0.4821 VLoss:0.4554 | TrAcc:0.797 VAcc:0.7
```

```
Epoch 4/30 | TrLoss:0.3670 VLoss:0.4056 | TrAcc:0.849 VAcc:0.8
```

```
Epoch 5/30 | TrLoss:0.3631 VLoss:0.3786 | TrAcc:0.836 VAcc:0.8
```

```
Epoch 6/30 | TrLoss:0.2861 VLoss:0.3932 | TrAcc:0.889 VAcc:0.7
```

```
Epoch 7/30 | TrLoss:0.2943 VLoss:0.4361 | TrAcc:0.875 VAcc:0.8
```

```
Epoch 8/30 | TrLoss:0.2832 VLoss:0.3943 | TrAcc:0.896 VAcc:0.8
```

```
Epoch 9/30 | TrLoss:0.2648 VLoss:0.4346 | TrAcc:0.893 VAcc:0.8
```

```
Epoch 10/30 | TrLoss:0.2419 VLoss:0.3906 | TrAcc:0.900 VAcc:0.8
```

```
Epoch 11/30 | TrLoss:0.2842 VLoss:0.4272 | TrAcc:0.881 VAcc:0.8
```

```
Epoch 12/30 | TrLoss:0.2123 VLoss:0.3200 | TrAcc:0.904 VAcc:0.8
```

```
Epoch 13/30 | TrLoss:0.1984 VLoss:0.3502 | TrAcc:0.924 VAcc:0.8
```

```
Epoch 14/30 | TrLoss:0.1871 VLoss:0.4228 | TrAcc:0.925 VAcc:0.8
```

```
Epoch 15/30 | TrLoss:0.1942 VLoss:0.3747 | TrAcc:0.921 VAcc:0.8
```

```
Epoch 16/30 | TrLoss:0.2057 VLoss:0.3747 | TrAcc:0.925 VAcc:0.8
```

```
Epoch 17/30 | TrLoss:0.2165 VLoss:0.2703 | TrAcc:0.910 VAcc:0.9
```

```
Epoch 18/30 | TrLoss:0.1754 VLoss:0.3644 | TrAcc:0.935 VAcc:0.8
```

```
Epoch 19/30 | TrLoss:0.1533 VLoss:0.3468 | TrAcc:0.944 VAcc:0.8
```

```
Epoch 20/30 | TrLoss:0.1542 VLoss:0.3743 | TrAcc:0.950 VAcc:0.8
```

```
Epoch 20/30 | TrLoss:0.1512 VLoss:0.3715 | TrAcc:0.920 VAcc:0.8
Epoch 21/30 | TrLoss:0.1897 VLoss:0.3695 | TrAcc:0.921 VAcc:0.8
Epoch 22/30 | TrLoss:0.1438 VLoss:0.3669 | TrAcc:0.943 VAcc:0.8
Epoch 23/30 | TrLoss:0.1637 VLoss:0.3329 | TrAcc:0.935 VAcc:0.8
Epoch 24/30 | TrLoss:0.1615 VLoss:0.3154 | TrAcc:0.940 VAcc:0.8
🛑 Early stopping at epoch 24
```

 Fold 1: Acc=0.921 Prec=0.875 Rec=0.991 F1=0.929 AUC=0.973

FOLD 2/5

```
Train: 731 | Val: 129 | Test: 215
Epoch 1/30 | TrLoss:0.6832 VLoss:0.6750 | TrAcc:0.551 VAcc:0.5
Epoch 2/30 | TrLoss:0.6107 VLoss:0.6158 | TrAcc:0.704 VAcc:0.6
Epoch 3/30 | TrLoss:0.4716 VLoss:0.4730 | TrAcc:0.811 VAcc:0.7
Epoch 4/30 | TrLoss:0.4047 VLoss:0.3748 | TrAcc:0.828 VAcc:0.8
Epoch 5/30 | TrLoss:0.3531 VLoss:0.4247 | TrAcc:0.864 VAcc:0.7
Epoch 6/30 | TrLoss:0.3210 VLoss:0.3415 | TrAcc:0.858 VAcc:0.8
Epoch 7/30 | TrLoss:0.2768 VLoss:0.4497 | TrAcc:0.883 VAcc:0.7
Epoch 8/30 | TrLoss:0.2357 VLoss:0.2953 | TrAcc:0.906 VAcc:0.8
Epoch 9/30 | TrLoss:0.2901 VLoss:0.3897 | TrAcc:0.879 VAcc:0.8
Epoch 10/30 | TrLoss:0.2710 VLoss:0.4662 | TrAcc:0.890 VAcc:0.8
Epoch 11/30 | TrLoss:0.2177 VLoss:0.3821 | TrAcc:0.912 VAcc:0.8
Epoch 12/30 | TrLoss:0.2156 VLoss:0.3210 | TrAcc:0.914 VAcc:0.8
Epoch 13/30 | TrLoss:0.2098 VLoss:0.5474 | TrAcc:0.912 VAcc:0.8
Epoch 14/30 | TrLoss:0.1606 VLoss:0.3269 | TrAcc:0.940 VAcc:0.8
Epoch 15/30 | TrLoss:0.1868 VLoss:0.3900 | TrAcc:0.933 VAcc:0.8
🛑 Early stopping at epoch 15
```

 Fold 2: Acc=0.907 Prec=0.876 Rec=0.955 F1=0.914 AUC=0.974

FOLD 3/5

```
Train: 731 | Val: 129 | Test: 215
Epoch 1/30 | TrLoss:0.6760 VLoss:0.6829 | TrAcc:0.607 VAcc:0.6
Epoch 2/30 | TrLoss:0.6003 VLoss:0.5569 | TrAcc:0.718 VAcc:0.7
Epoch 3/30 | TrLoss:0.4820 VLoss:0.4219 | TrAcc:0.808 VAcc:0.8
Epoch 4/30 | TrLoss:0.3864 VLoss:0.4013 | TrAcc:0.853 VAcc:0.8
Epoch 5/30 | TrLoss:0.3203 VLoss:0.3862 | TrAcc:0.860 VAcc:0.8
Epoch 6/30 | TrLoss:0.3024 VLoss:0.3197 | TrAcc:0.872 VAcc:0.8
Epoch 7/30 | TrLoss:0.3237 VLoss:0.3871 | TrAcc:0.871 VAcc:0.8
Epoch 8/30 | TrLoss:0.2794 VLoss:0.3678 | TrAcc:0.881 VAcc:0.8
Epoch 9/30 | TrLoss:0.2679 VLoss:0.3431 | TrAcc:0.901 VAcc:0.8
Epoch 10/30 | TrLoss:0.2589 VLoss:0.2602 | TrAcc:0.896 VAcc:0.8
Epoch 11/30 | TrLoss:0.2445 VLoss:0.2954 | TrAcc:0.903 VAcc:0.8
Epoch 12/30 | TrLoss:0.2334 VLoss:0.4095 | TrAcc:0.908 VAcc:0.7
Epoch 13/30 | TrLoss:0.2006 VLoss:0.2501 | TrAcc:0.919 VAcc:0.8
Epoch 14/30 | TrLoss:0.1783 VLoss:0.3801 | TrAcc:0.933 VAcc:0.8
Epoch 15/30 | TrLoss:0.1942 VLoss:0.3470 | TrAcc:0.925 VAcc:0.8
Epoch 16/30 | TrLoss:0.2136 VLoss:0.3543 | TrAcc:0.922 VAcc:0.8
Epoch 17/30 | TrLoss:0.2033 VLoss:0.3120 | TrAcc:0.926 VAcc:0.8
Epoch 18/30 | TrLoss:0.1513 VLoss:0.3385 | TrAcc:0.943 VAcc:0.8
Epoch 19/30 | TrLoss:0.1661 VLoss:0.3184 | TrAcc:0.933 VAcc:0.8
```

Epoch 19/30 | TrLoss:0.1001 VLoss:0.3107 | TrAcc:0.933 VlAcc:0.8
 Epoch 20/30 | TrLoss:0.1346 VLoss:0.3552 | TrAcc:0.949 VlAcc:0.8
 Early stopping at epoch 20

 Fold 3: Acc=0.907 Prec=0.862 Rec=0.982 F1=0.918 AUC=0.968

FOLD 4/5

Train: 731 | Val: 129 | Test: 215
 Epoch 1/30 | TrLoss:0.6777 VLoss:0.6743 | TrAcc:0.581 VlAcc:0.5
 Epoch 2/30 | TrLoss:0.6062 VLoss:0.5537 | TrAcc:0.728 VlAcc:0.7
 Epoch 3/30 | TrLoss:0.4694 VLoss:0.4852 | TrAcc:0.817 VlAcc:0.8
 Epoch 4/30 | TrLoss:0.3820 VLoss:0.5366 | TrAcc:0.850 VlAcc:0.7
 Epoch 5/30 | TrLoss:0.3633 VLoss:0.4118 | TrAcc:0.864 VlAcc:0.8
 Epoch 6/30 | TrLoss:0.3057 VLoss:0.3899 | TrAcc:0.875 VlAcc:0.8
 Epoch 7/30 | TrLoss:0.3012 VLoss:0.4967 | TrAcc:0.869 VlAcc:0.8
 Epoch 8/30 | TrLoss:0.2764 VLoss:0.3420 | TrAcc:0.887 VlAcc:0.8
 Epoch 9/30 | TrLoss:0.2649 VLoss:0.3628 | TrAcc:0.893 VlAcc:0.8
 Epoch 10/30 | TrLoss:0.2628 VLoss:0.5795 | TrAcc:0.894 VlAcc:0.7
 Epoch 11/30 | TrLoss:0.2332 VLoss:0.5651 | TrAcc:0.917 VlAcc:0.7
 Epoch 12/30 | TrLoss:0.2085 VLoss:0.5058 | TrAcc:0.910 VlAcc:0.8
 Epoch 13/30 | TrLoss:0.2187 VLoss:0.4577 | TrAcc:0.907 VlAcc:0.7
 Epoch 14/30 | TrLoss:0.2058 VLoss:0.4977 | TrAcc:0.925 VlAcc:0.8
 Epoch 15/30 | TrLoss:0.1763 VLoss:0.3867 | TrAcc:0.929 VlAcc:0.8
 Early stopping at epoch 15

 Fold 4: Acc=0.879 Prec=0.827 Rec=0.973 F1=0.894 AUC=0.958

FOLD 5/5

Train: 731 | Val: 129 | Test: 215
 Epoch 1/30 | TrLoss:0.6802 VLoss:0.6847 | TrAcc:0.532 VlAcc:0.5
 Epoch 2/30 | TrLoss:0.5975 VLoss:0.6335 | TrAcc:0.704 VlAcc:0.7
 Epoch 3/30 | TrLoss:0.4946 VLoss:0.5487 | TrAcc:0.803 VlAcc:0.8
 Epoch 4/30 | TrLoss:0.3976 VLoss:0.4172 | TrAcc:0.847 VlAcc:0.8
 Epoch 5/30 | TrLoss:0.3758 VLoss:0.4684 | TrAcc:0.847 VlAcc:0.8
 Epoch 6/30 | TrLoss:0.2843 VLoss:0.3647 | TrAcc:0.889 VlAcc:0.8
 Epoch 7/30 | TrLoss:0.2842 VLoss:0.4194 | TrAcc:0.872 VlAcc:0.7
 Epoch 8/30 | TrLoss:0.2831 VLoss:0.3969 | TrAcc:0.876 VlAcc:0.8
 Epoch 9/30 | TrLoss:0.2306 VLoss:0.3690 | TrAcc:0.912 VlAcc:0.8
 Epoch 10/30 | TrLoss:0.2212 VLoss:0.4571 | TrAcc:0.906 VlAcc:0.8
 Epoch 11/30 | TrLoss:0.2318 VLoss:0.5011 | TrAcc:0.906 VlAcc:0.7
 Epoch 12/30 | TrLoss:0.1640 VLoss:0.3777 | TrAcc:0.940 VlAcc:0.8
 Epoch 13/30 | TrLoss:0.2276 VLoss:0.3267 | TrAcc:0.914 VlAcc:0.8
 Epoch 14/30 | TrLoss:0.1852 VLoss:0.4648 | TrAcc:0.931 VlAcc:0.7
 Epoch 15/30 | TrLoss:0.1815 VLoss:0.2542 | TrAcc:0.932 VlAcc:0.8
 Epoch 16/30 | TrLoss:0.1645 VLoss:0.3991 | TrAcc:0.922 VlAcc:0.8
 Epoch 17/30 | TrLoss:0.1912 VLoss:0.3203 | TrAcc:0.925 VlAcc:0.8
 Epoch 18/30 | TrLoss:0.1619 VLoss:0.3980 | TrAcc:0.938 VlAcc:0.8
 Epoch 19/30 | TrLoss:0.1826 VLoss:0.3719 | TrAcc:0.931 VlAcc:0.8
 Epoch 20/30 | TrLoss:0.1566 VLoss:0.5242 | TrAcc:0.940 VlAcc:0.7
 Epoch 21/30 | TrLoss:0.1682 VLoss:0.4732 | TrAcc:0.921 VlAcc:0.8
 Epoch 22/30 | TrLoss:0.1522 VLoss:0.4124 | TrAcc:0.940 VlAcc:0.8

EPOCH 22/30 | 11LOSS:0.1933 11LOSS:0.4134 | 11ACC:0.940 11ACC:0.8



Early stopping at epoch 22



Fold 5: Acc=0.930 Prec=0.922 Rec=0.947 F1=0.934 AUC=0.960



All folds completed!



EXPERIMENT 2b: IMAGE (NO BG) — ALL BRANDS — RESULTS (5-Fold CV)

Fold	Accuracy	Precision	Recall	F1-Score	AUC-ROC
1	0.9209	0.8750	0.9912	0.9295	0.9728
2	0.9070	0.8760	0.9550	0.9138	0.9737
3	0.9070	0.8615	0.9825	0.9180	0.9681
4	0.8791	0.8271	0.9735	0.8943	0.9584
5	0.9302	0.9224	0.9469	0.9345	0.9599
Mean	0.9088	0.8724	0.9698	0.9180	0.9666
Std	0.0193	0.0343	0.0185	0.0157	0.0071

	Fold	Accuracy	Precision	Recall	F1	AUC
0	1	0.9209	0.8750	0.9912	0.9295	0.9728
1	2	0.9070	0.8760	0.9550	0.9138	0.9737
2	3	0.9070	0.8615	0.9825	0.9180	0.9681
3	4	0.8791	0.8271	0.9735	0.8943	0.9584
4	5	0.9302	0.9224	0.9469	0.9345	0.9599

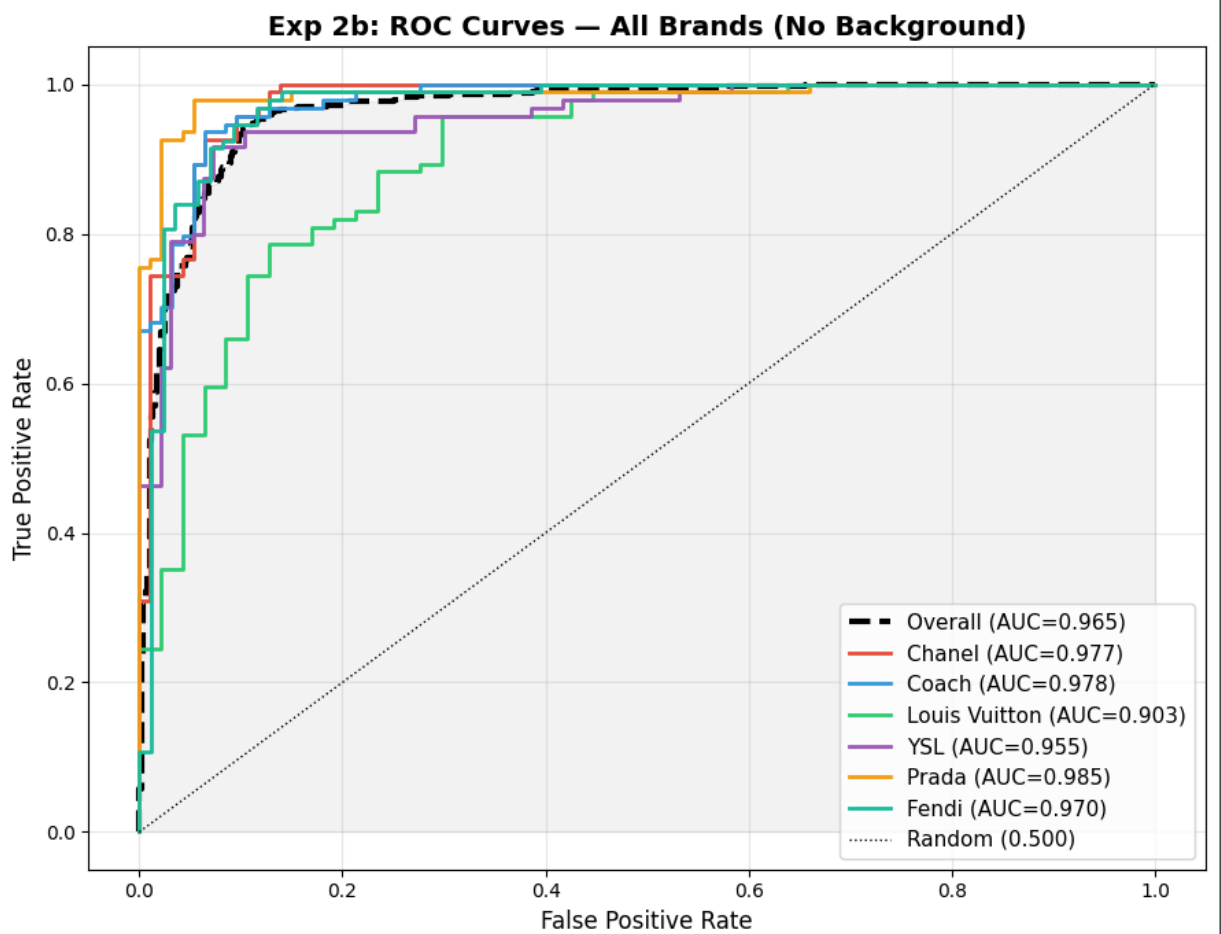
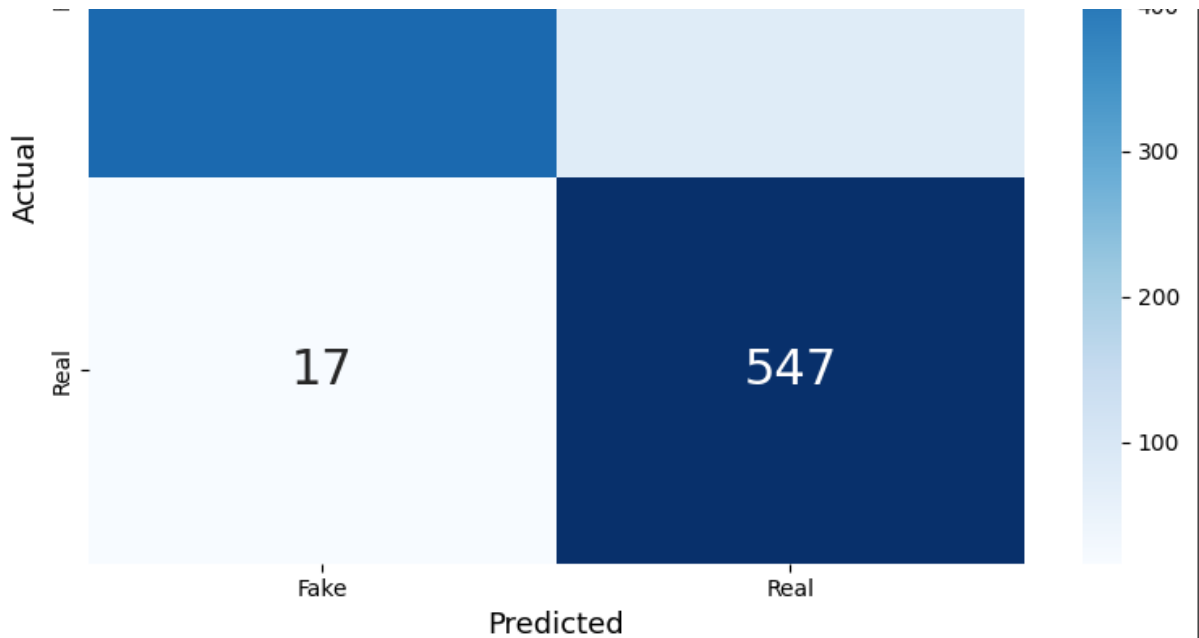


PER-BRAND BREAKDOWN (5-Fold Aggregate)

Brand	Acc	Prec	Rec	F1	AUC	
Chanel	0.9149	0.8545	1.0000	0.9216	0.9771	1
Coach	0.9309	0.9091	0.9574	0.9326	0.9780	1
Louis Vuitton	0.8440	0.8333	0.9574	0.8911	0.9031	1
YSL	0.9005	0.8725	0.9368	0.9036	0.9552	1
Prada	0.9309	0.8932	0.9787	0.9340	0.9852	1
Fendi	0.9162	0.8679	0.9892	0.9246	0.9702	1

Exp 2b (All Brands): Images (No Background) — EfficientNet-B0
5-Fold CV | Acc=0.909 | F1=0.918







Saved: exp2b_allbrands_training_history.png



FILES SAVED:



exp2b_allbrands_results.csv / .xlsx
exp2b_allbrands_confusion_matrix.png
exp2b_allbrands_roc_curve.png
exp2b_allbrands_training_history.png



=====

EXPERIMENT 2b (ALL BRANDS) COMPLETE!

=====



Overall: Acc=0.9088 F1=0.9180 AUC=0.9666

